DEPLOYMENT OF 3-TIER APPLICATIONS USING AWS SERVICES

A Traditional Cloud Deployment for a Web Application with Database Backend

---

Domain

Cloud Computing and DevOps

Prepared by: Parikshit A

Cloud Provider: Amazon Web Services (AWS)

Region: ap-south-1 (Asia Pacific — Mumbai)

Date: August 2026

Table of Contents

1. Project Overview 3
2. Architecture & Key Components 4
3. Networking & Security Foundation (VPC) 5
4. Presentation Tier — S3 Static Website 6
5. Application Tier — EC2 Backend 10
6. Database Tier — Amazon RDS 12
7. Identity & Access Management (IAM) 14
8. DNS & Custom Domain (Route 53) 16
9. HTTPS & Content Delivery (ACM / CloudFront) 18
10. End-to-End Testing & Verification 20
11. Summary & Outcomes 22

1. Project Overview

This project demonstrates a manually deployed, traditional 3-tier architecture on AWS for a web application that accepts user-submitted data and stores it in a backend database. Unlike the Zomato deployment, this setup deliberately avoids DevOps tooling — no Jenkins, no Docker, no Kubernetes — and relies entirely on native AWS services for hosting, networking, and storage.

The goal is to show how a classic web application can be assembled directly from AWS building blocks: a static frontend, an application server, and a managed database, tied together with careful networking and IAM configuration.

1.1 Source Repository

GitHub: github.com/staragile2016/AWS-3-Tier-Architecture

1.2 Key Components

- Presentation Tier (Frontend) — Hosted on AWS S3 as a static website.
- Application Tier (Backend) — Runs on an AWS EC2 web server (Node.js behind Nginx).
- Database Tier — Managed by AWS RDS (MySQL).
- Networking & Security — VPC, IAM, and Route 53 govern access control and DNS.

1.3 Business Requirements

- Users should be able to submit data via a web form.
- Data should be stored securely in a backend database.
- The application should be accessible via a custom domain (using Route 53).
- Security best practices should be followed — IAM roles, VPC isolation.

2. Architecture & Key Components

Each tier maps to a purpose-built AWS service, with traffic flowing from the browser, through the static frontend, into the application server, and finally to the managed database.

Tier	AWS Service	Role
Presentation	Amazon S3 (static website hosting)	Serves HTML/CSS/JS to the browser
Application	Amazon EC2 (Nginx + Node.js)	Processes requests, talks to the database
Database	Amazon RDS (MySQL)	Persists submitted data
Networking	Amazon VPC, Security Groups	Isolates and secures each tier
Identity	AWS IAM	Grants least-privilege access between services
DNS	Amazon Route 53	Maps a custom domain to the application
TLS / CDN (optional)	ACM, CloudFront	HTTPS and edge delivery for the frontend

Table 2.1 — Mapping of application tiers to AWS services.

2.2 Request Flow

- A user visits the custom domain, resolved by Route 53 to the S3 static website (optionally via CloudFront).
- The React frontend loaded from S3 calls the EC2-hosted API (e.g. a /submit-data or /transaction endpoint).
- The EC2 application server validates the request and writes to the RDS MySQL database.
- Security Groups ensure only the intended tiers can reach each other — RDS accepts connections only from the EC2 security group, and the database itself lives in a private subnet with no direct internet exposure.

3. Networking & Security Foundation (VPC)

A dedicated VPC gives the application network isolation, with public and private subnets separating internet-facing resources from the database.

VPC Name	3Tire-Application
VPC ID	vpc-019793faf1454c393
Status	Available
Subnets	4 — Public-Subnet & Private-Subnet in ap-south-1a, Public-Subnet & private-subnet in ap-south-1b
Route Table	RT-1 (rtb-05ee21ff1fc89cf58)
Internet Gateway	IGW — provides outbound/inbound internet access to the public subnets

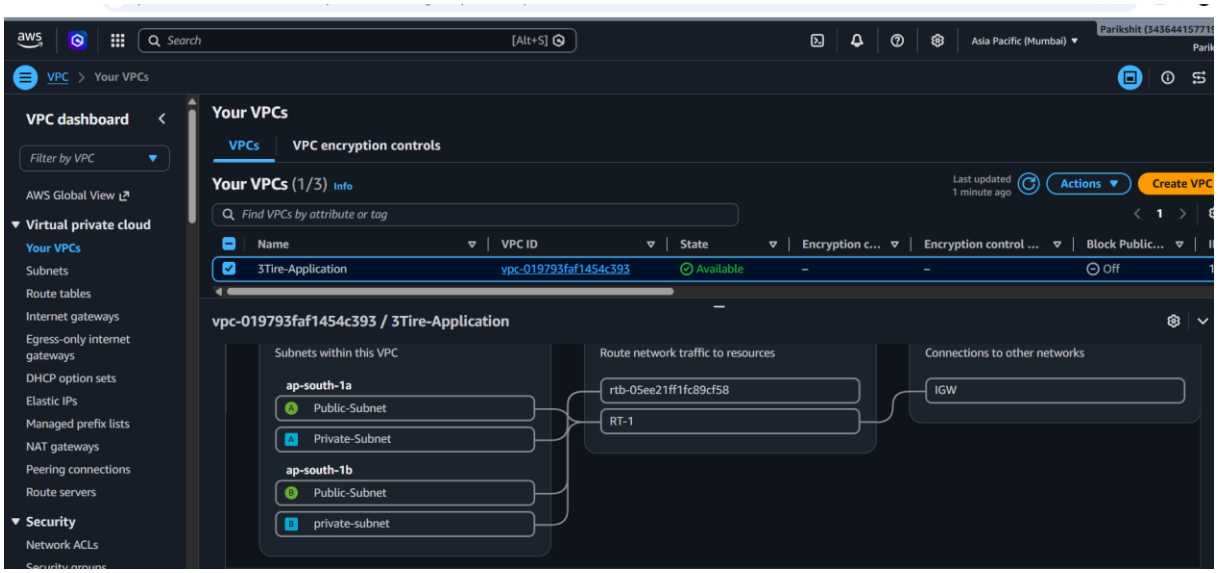


Figure 3.1 — VPC resource map: public/private subnets across two Availability Zones, route table, and Internet Gateway.

The EC2 application server sits in a public subnet so it can be reached over the internet, while RDS lives in a private subnet, reachable only from within the VPC. Spreading subnets across ap-south-1a and ap-south-1b also gives the design a foundation for future multi-AZ resilience.

4. Presentation Tier — S3 Static Website

The frontend is a React single-page application, built locally and published straight to an S3 bucket configured for static website hosting.

4.1 Frontend Source

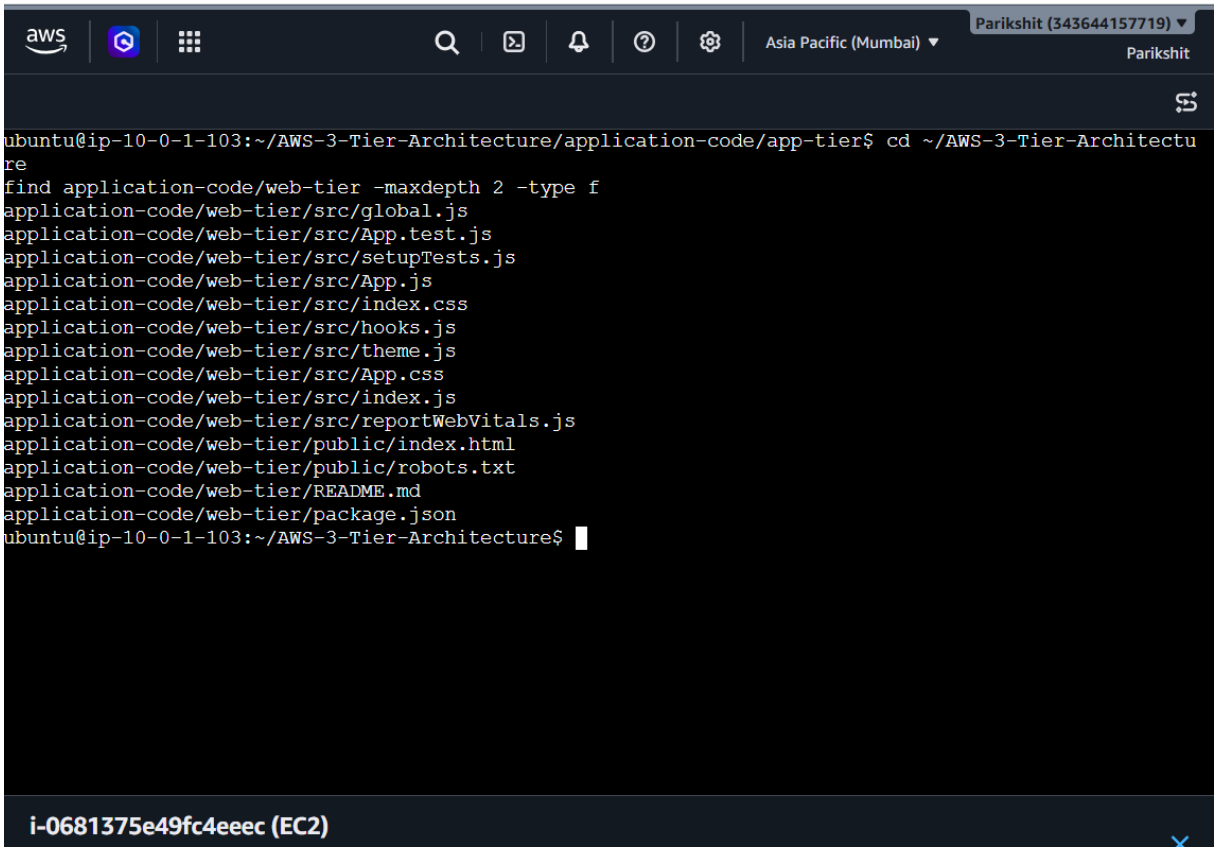


Figure 4.1 — React frontend source layout (application-code/web-tier) on the build server.

4.2 Production Build

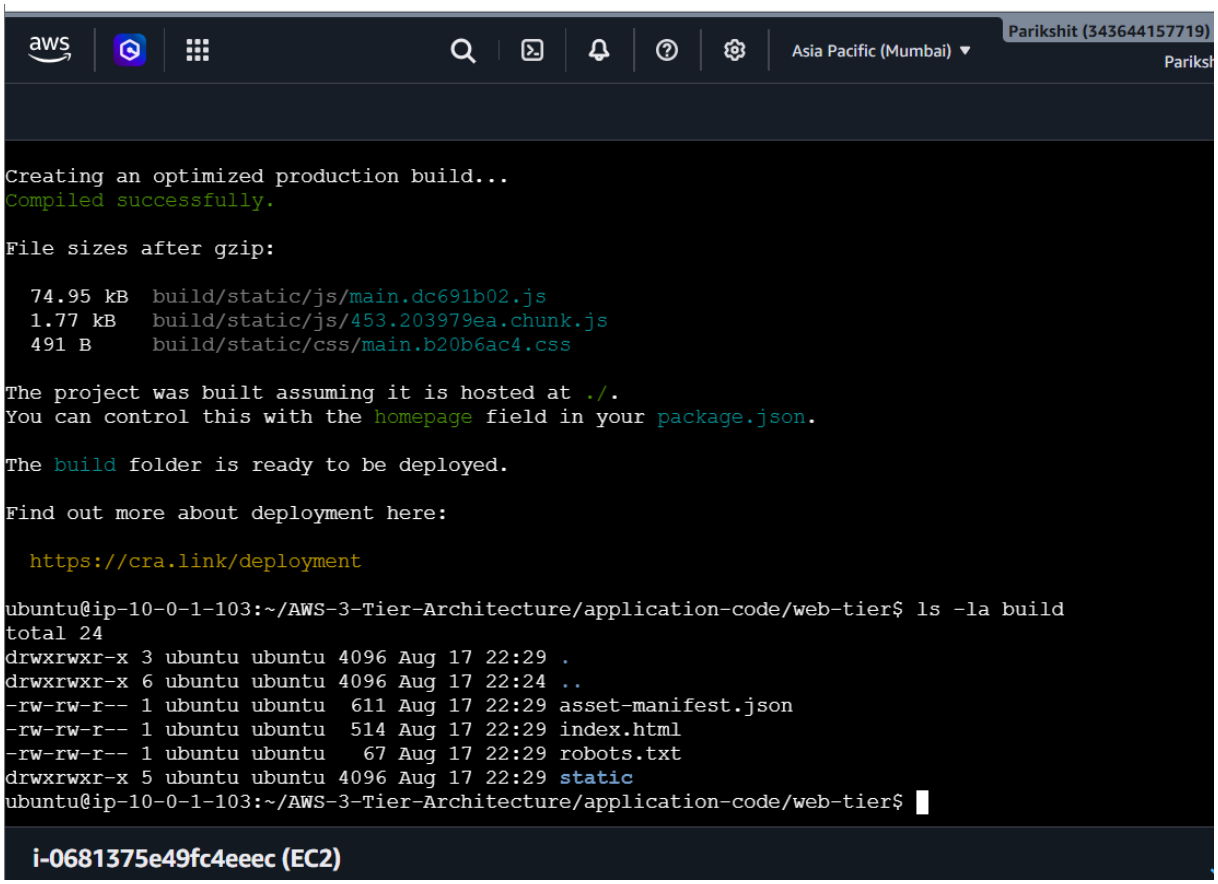


Figure 4.2 — Optimized production build compiled successfully; output written to the build/ folder.

4.3 Publishing to S3

The compiled build/ folder is synced directly to the S3 bucket using the AWS CLI, authenticated via an IAM role assumed on the EC2 build host.

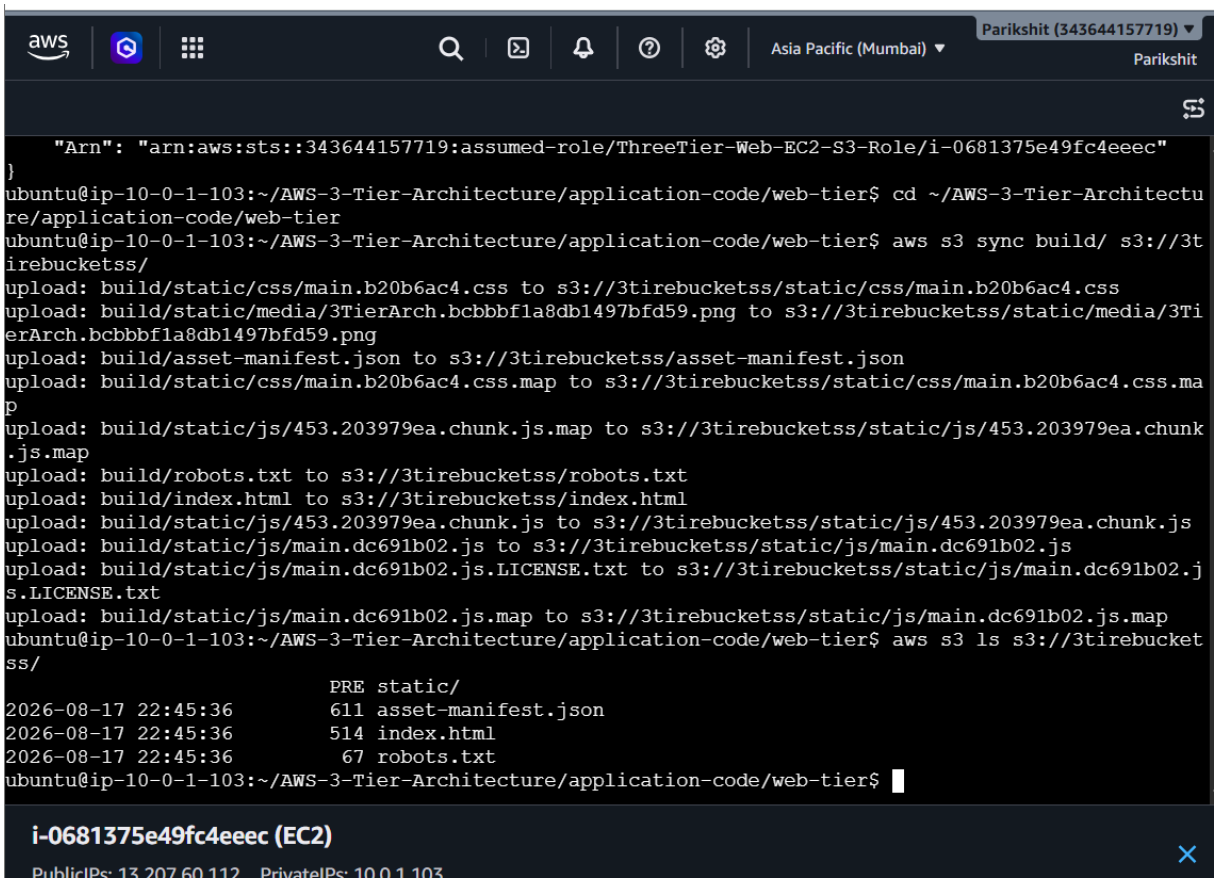


Figure 4.3 — aws s3 sync uploading the production build to s3://3tirebucketss.

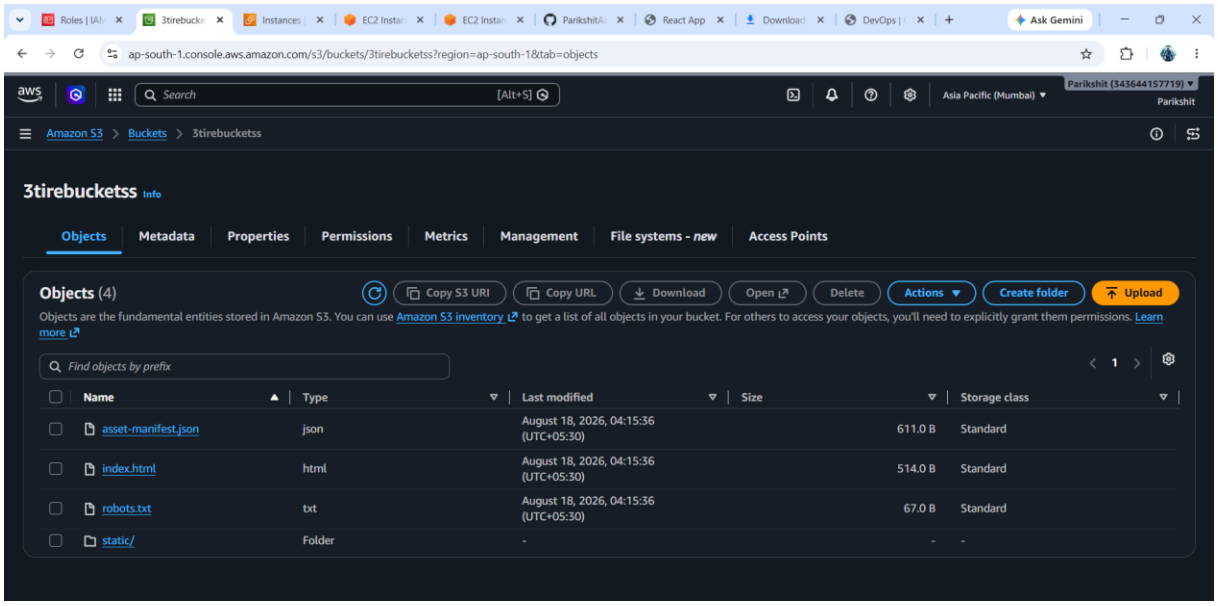


Figure 4.4 — Resulting objects in the 3tirebucketss S3 bucket: index.html, static assets, and manifest.

4.4 Public Read Access

A bucket policy grants public GetObject access so the static site can be served directly to browsers over the S3 website endpoint.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::3tirebucketss/*"
    }
  ]
}
```

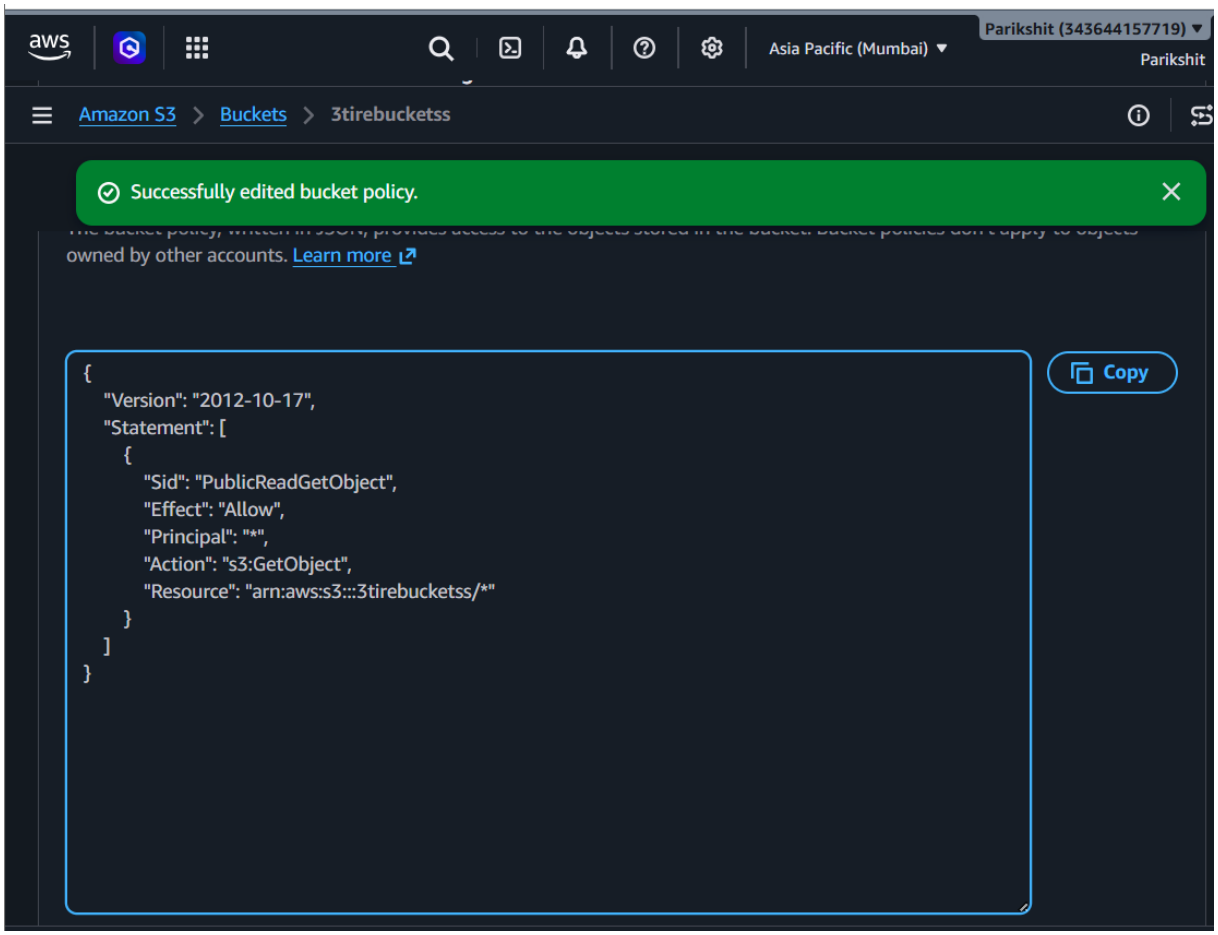


Figure 4.5 — Bucket policy granting public read access for static website hosting.

5. Application Tier — EC2 Backend

A single EC2 instance runs the backend API behind Nginx as a reverse proxy. The application listens on port 4000 and exposes health-check and transaction endpoints that the frontend calls.

Instance ID	i-0681375e49fc4eeec
Instance Type	t3.micro
Public IP	13.207.60.112
Private IP	10.0.1.103
Web Server	Nginx (reverse proxy) — active and running
Backend Runtime	Node.js app listening on http://localhost:4000

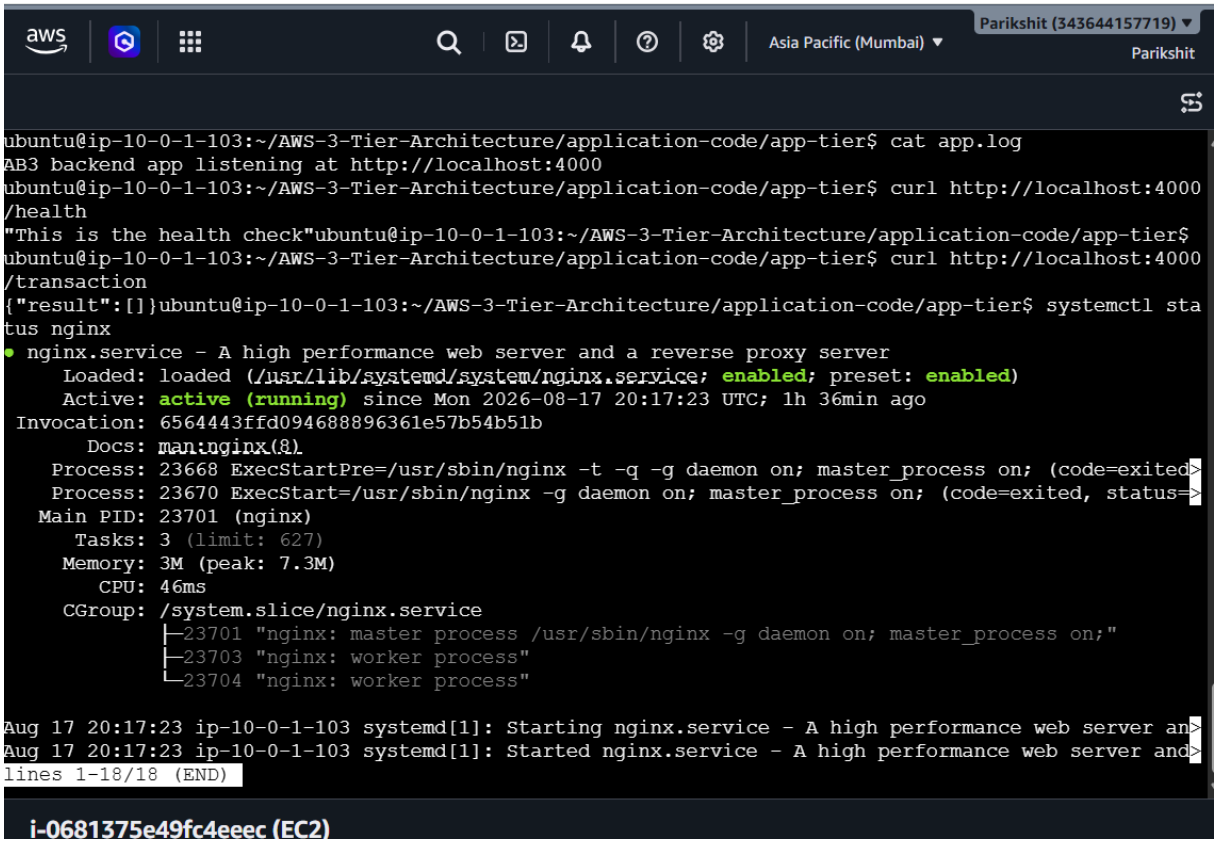


Figure 5.1 — Backend health checks (/health, /transaction) responding correctly; Nginx service active and running on the EC2 instance.

5.1 Network Access Control

A dedicated security group, Three-Tire-EC2, controls exactly what traffic can reach the application server.

Type	Protocol	Port	Purpose
SSH	TCP	22	Administration

Type	Protocol	Port	Purpose
Custom TCP	TCP	4000	Backend API
HTTPS	TCP	443	Secure web traffic
HTTP	TCP	80	Web traffic (Nginx)

Table 5.1 — Inbound rules on the Three-Tire-EC2 security group.

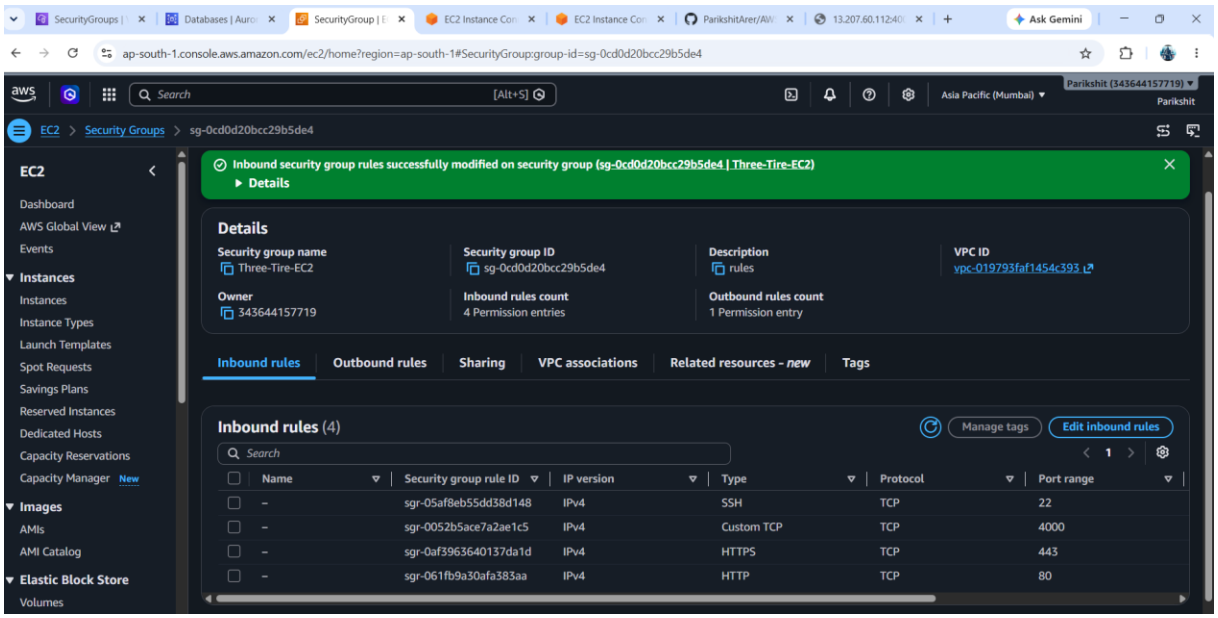


Figure 5.2 — Three-Tire-EC2 security group inbound rules.

6. Database Tier — Amazon RDS

A managed MySQL instance on Amazon RDS stores the application's data, isolated in the private subnet and reachable only from the application tier.

DB Identifier	database-2
Engine	MySQL
Instance Class	db.t4g.micro
Availability Zone	ap-south-1a
Status	Available

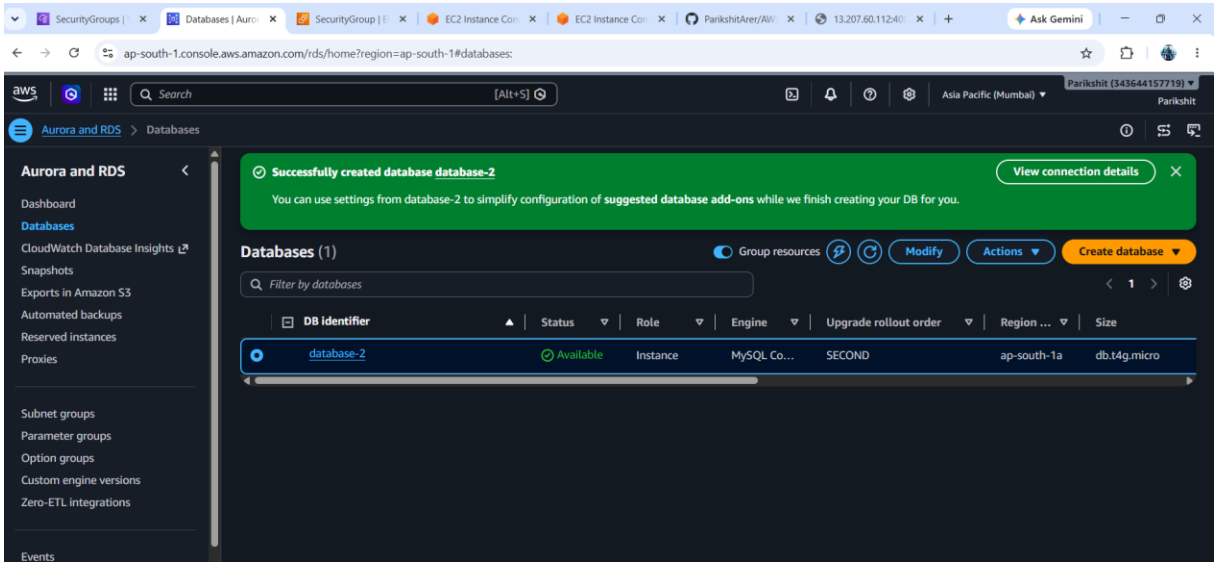


Figure 6.1 — Amazon RDS database instance "database-2" successfully created and available.

6.1 Schema

The application uses a webappdb database with a single transactions table.

Field	Type	Null	Key	Extra
id	int	NO	PRI	auto_increment
amount	decimal(10,2)	YES	—	—
description	varchar(255)	YES	—	—

Table 6.1 — Structure of the webappdb.transactions table.

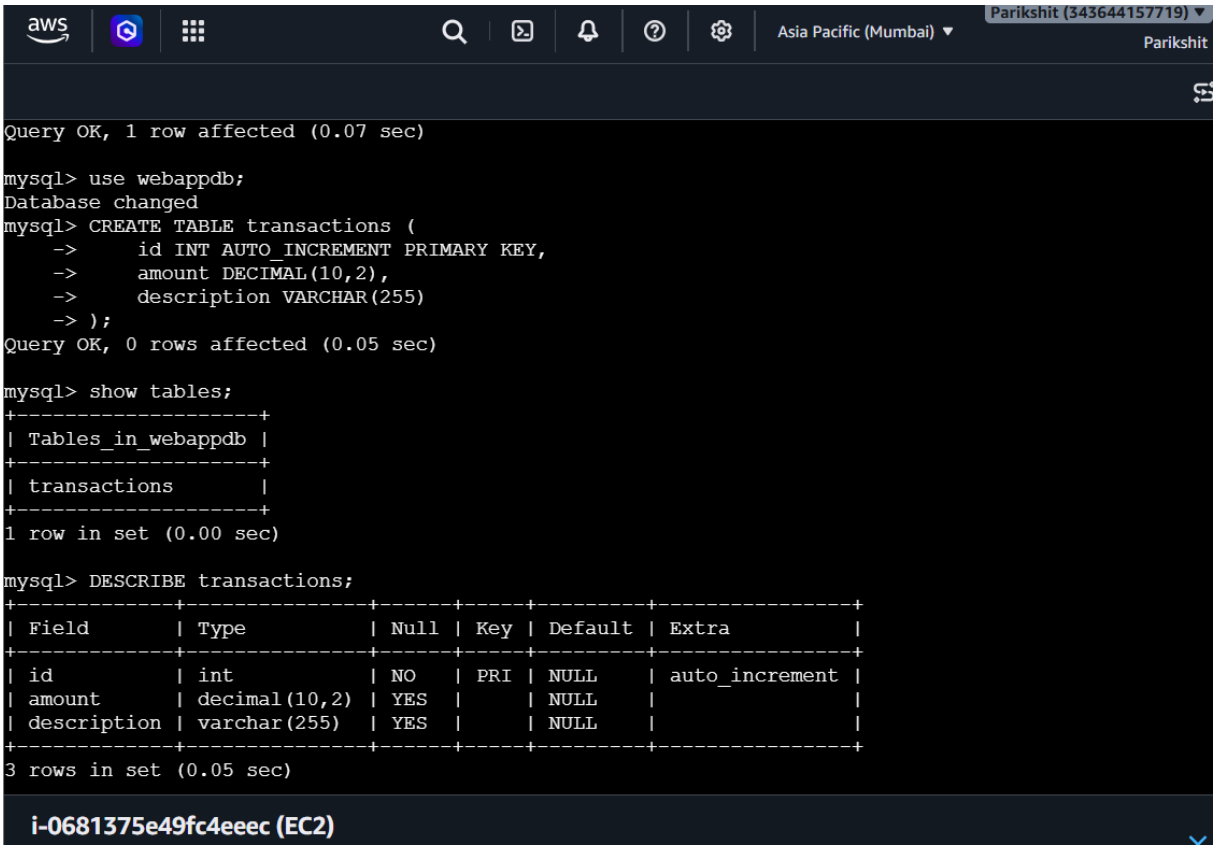


Figure 6.2 — Creating the webappdb database and transactions table via the MySQL client from the application server.

6.2 Network Access Control

The Three-Tire-RDS security group permits inbound MySQL/Aurora traffic (port 3306) only from within the VPC — in practice, from the EC2 application tier.

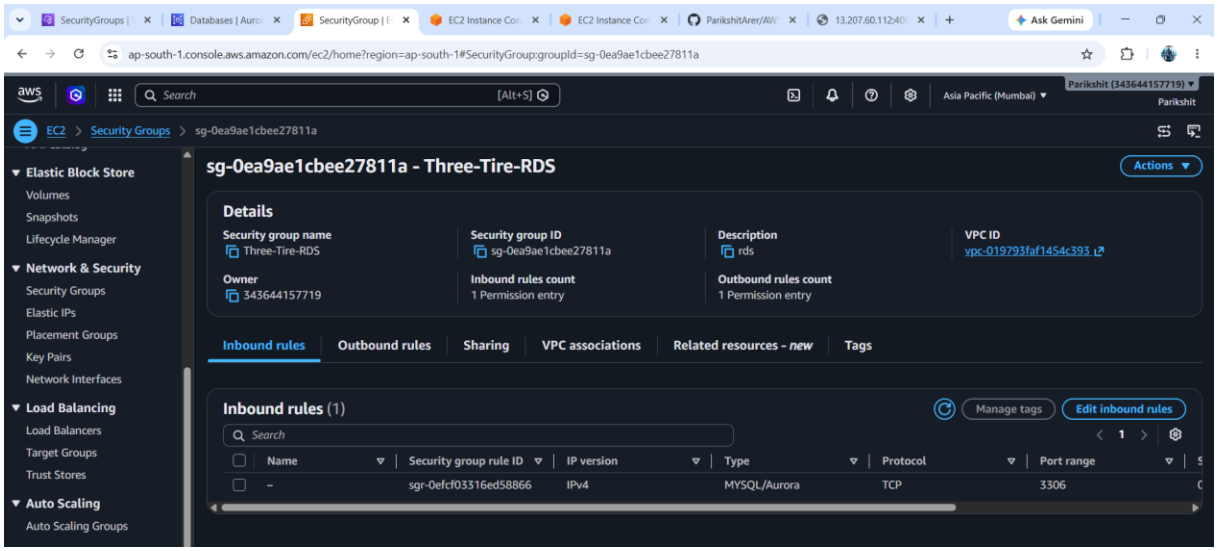


Figure 6.3 — Three-Tire-RDS security group: inbound MySQL/Aurora traffic on port 3306.

7. Identity & Access Management (IAM)

Rather than embedding long-lived AWS credentials on the EC2 instance, the application server assumes an IAM role that grants exactly the permissions it needs — in this case, the ability to upload the compiled frontend build to S3.

Role Name	ThreeTier-Web-EC2-S3-Role
Purpose	Allows the EC2 instance to sync the React build to the S3 bucket
Attached To	EC2 instance i-0681375e49fc4eeec

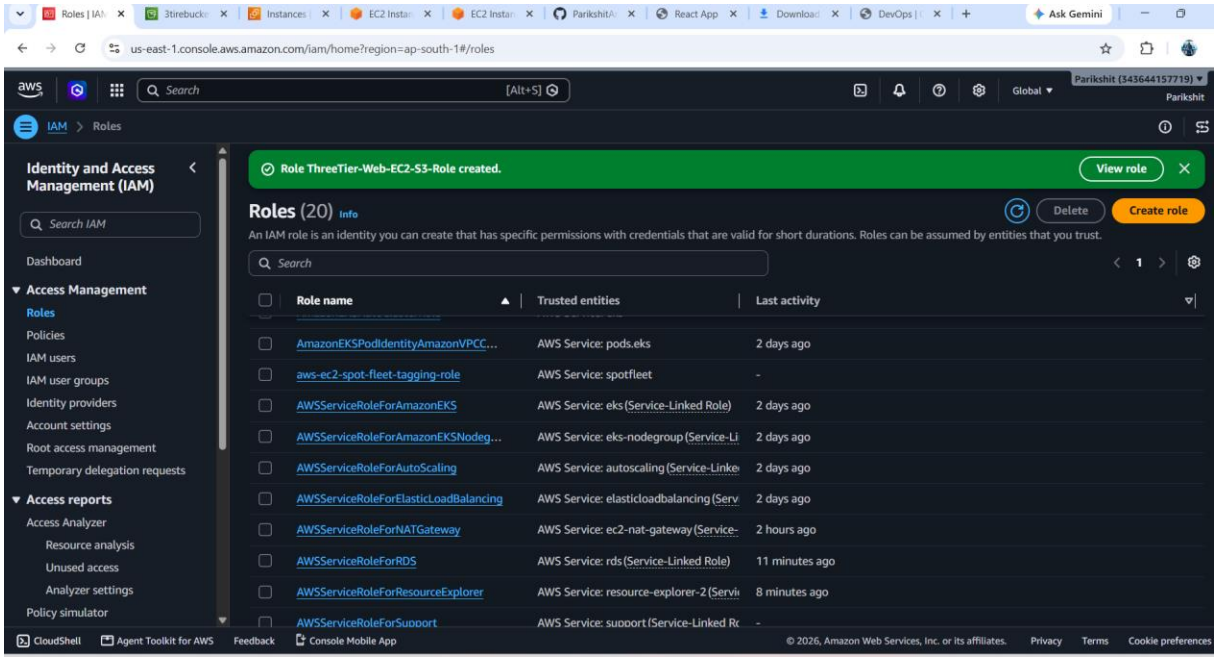


Figure 7.1 — ThreeTier-Web-EC2-S3-Role created in IAM.

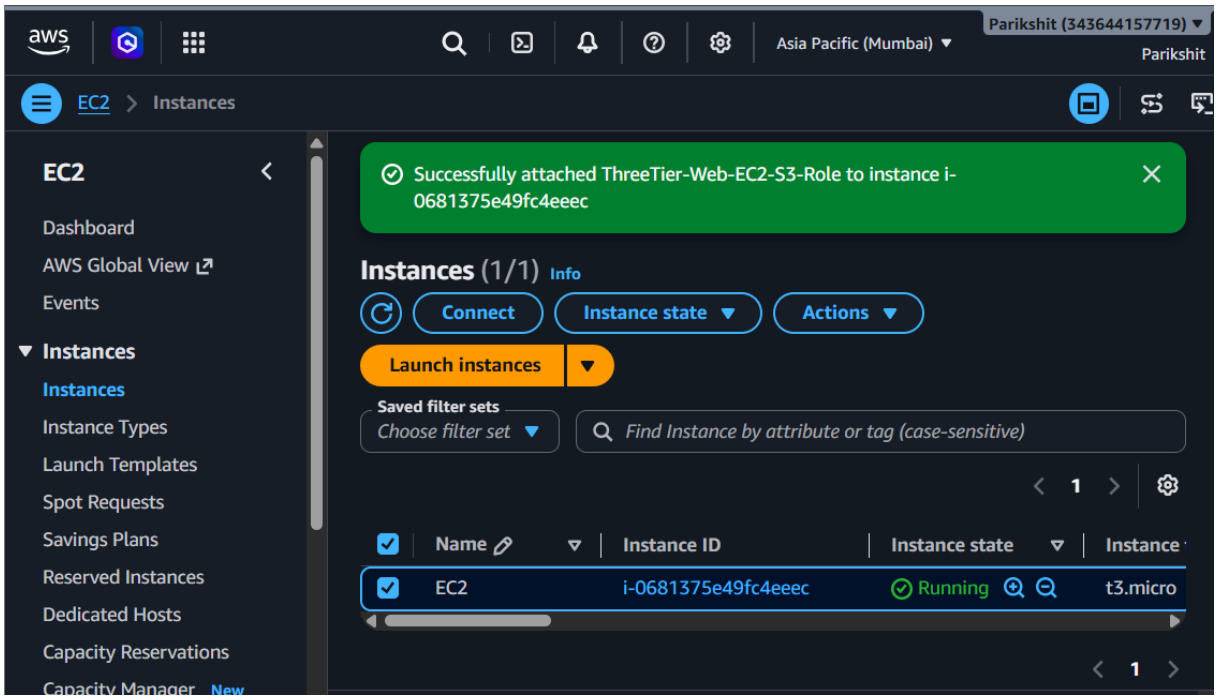


Figure 7.2 — Role successfully attached to the EC2 instance, enabling aws s3 sync without stored credentials.

8. DNS & Custom Domain (Route 53)

A public hosted zone in Route 53 maps a human-friendly domain to the application, satisfying the requirement for custom-domain access.

Hosted Zone	parikshitarer.in (Public)
Record Types	NS, SOA (zone defaults) + A record
A Record Target	3tirebucketss.s3-website.ap-south-1.amazonaws.com
Routing Policy	Simple routing
TTL	300 seconds

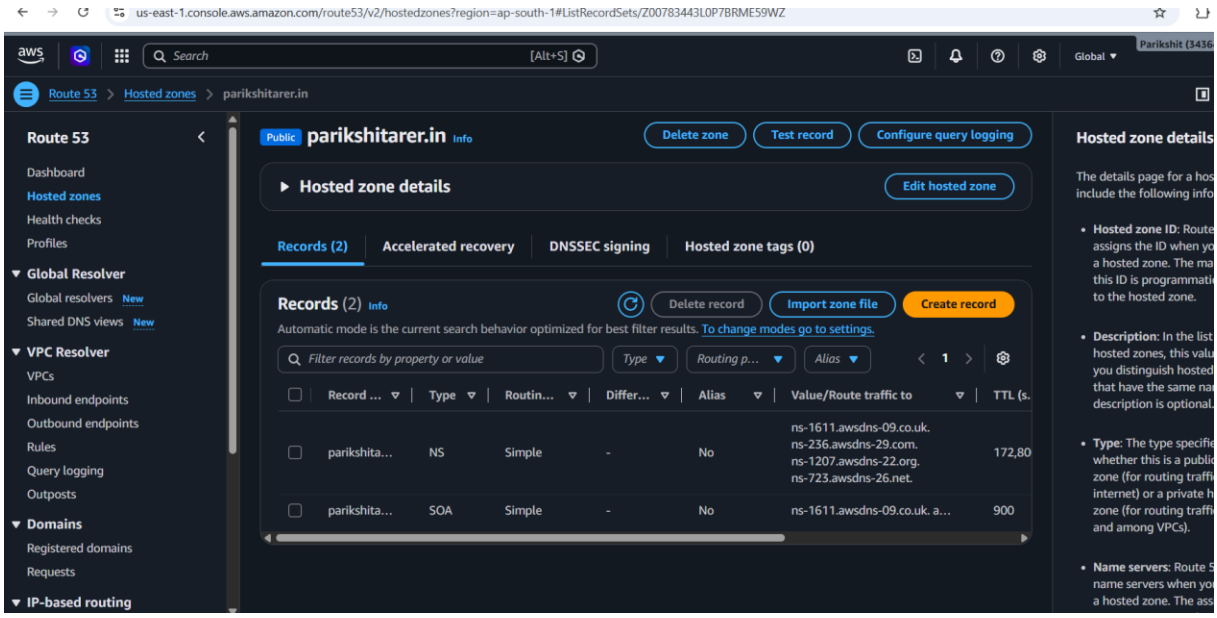


Figure 8.1 — Route 53 public hosted zone for parikshitarer.in.

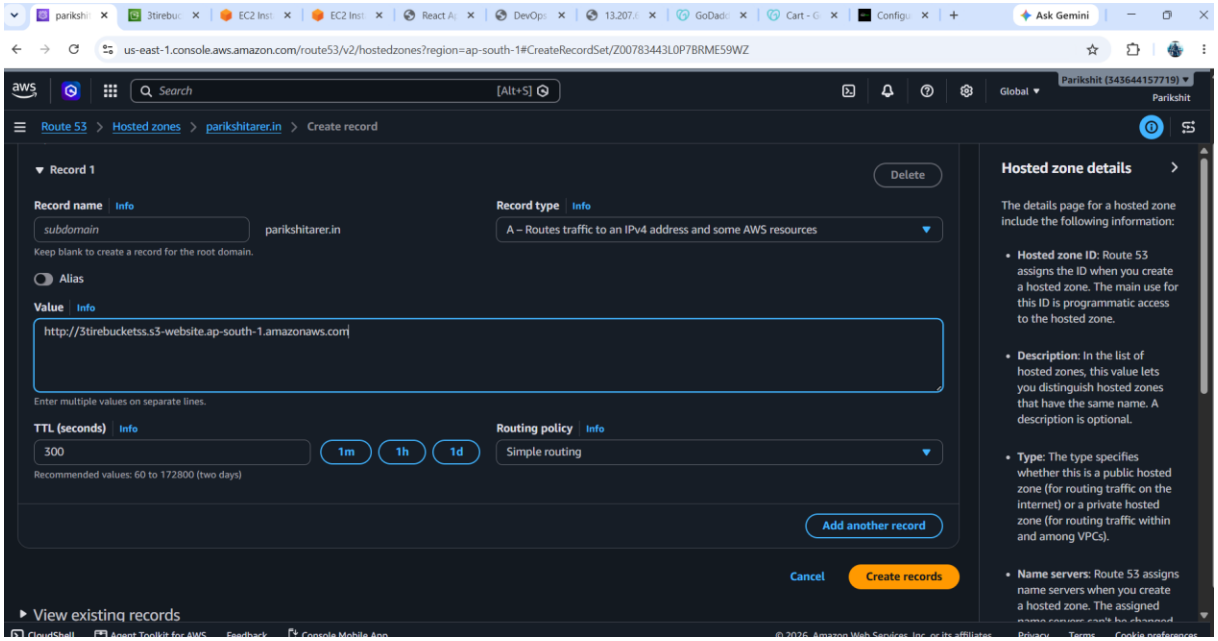


Figure 8.2 — Creating an A record that points the root domain at the S3 static-website endpoint.

9. HTTPS & Content Delivery (ACM / CloudFront)

To serve the site over HTTPS on the custom domain, a public certificate was requested through AWS Certificate Manager, and a CloudFront distribution was configured in front of the S3 origin — S3 website endpoints do not support HTTPS directly, so CloudFront is the standard way to add TLS in front of a static site.

9.1 Certificate Request

Certificate ID	a545685b-c6db-4d66-91da-b514560f4c50
Domains Covered	parikshitarer.in, www.parikshitarer.in
Validation Method	DNS
Status	Pending validation
Key Type	RSA 2048

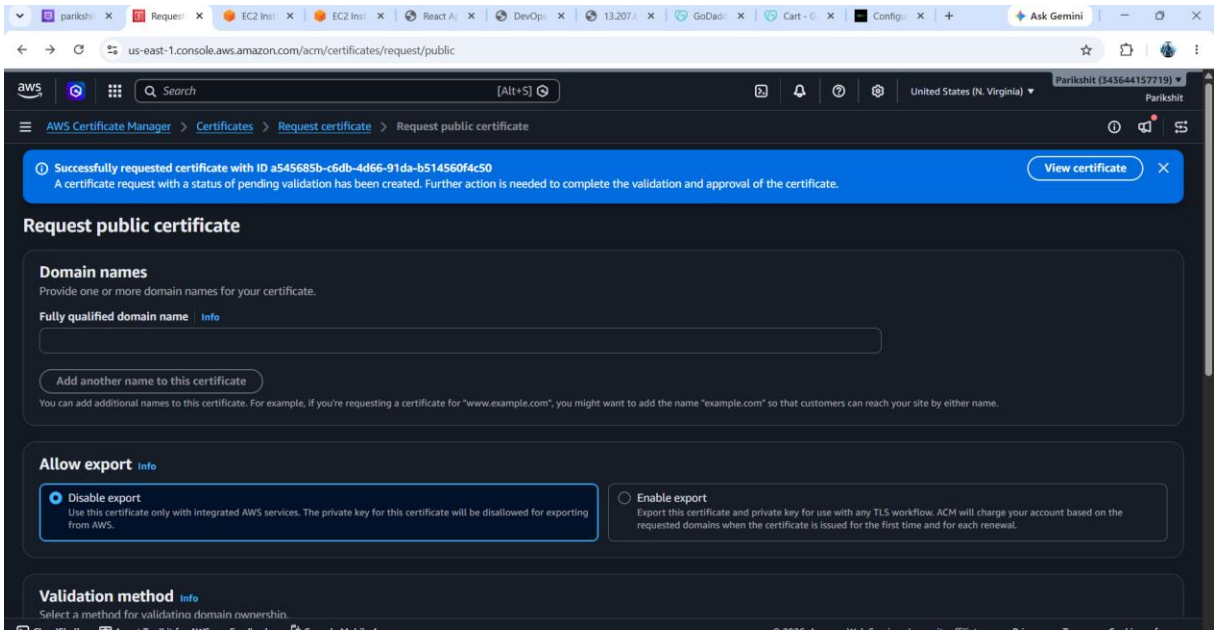


Figure 9.1 — Requesting a public ACM certificate for the custom domain.

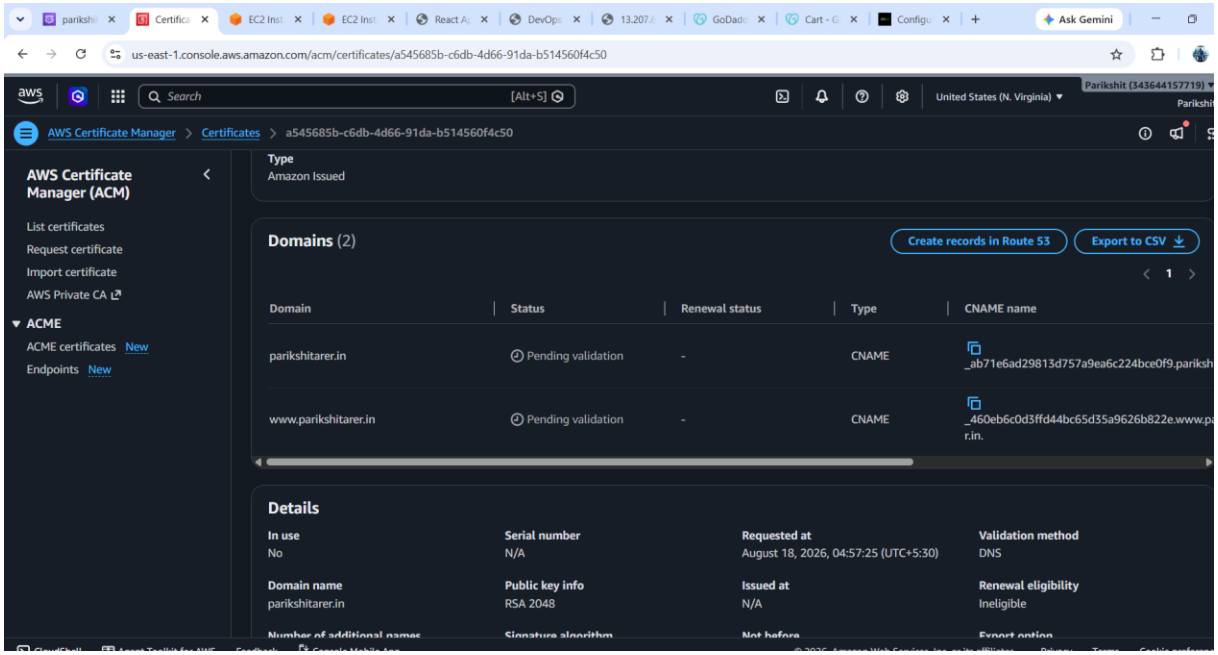


Figure 9.2 — Certificate details showing both domains awaiting DNS validation.

9.2 CloudFront Distribution

A CloudFront distribution was created with the S3 static website endpoint as its origin, positioning CloudFront to terminate HTTPS and cache content at edge locations close to users.

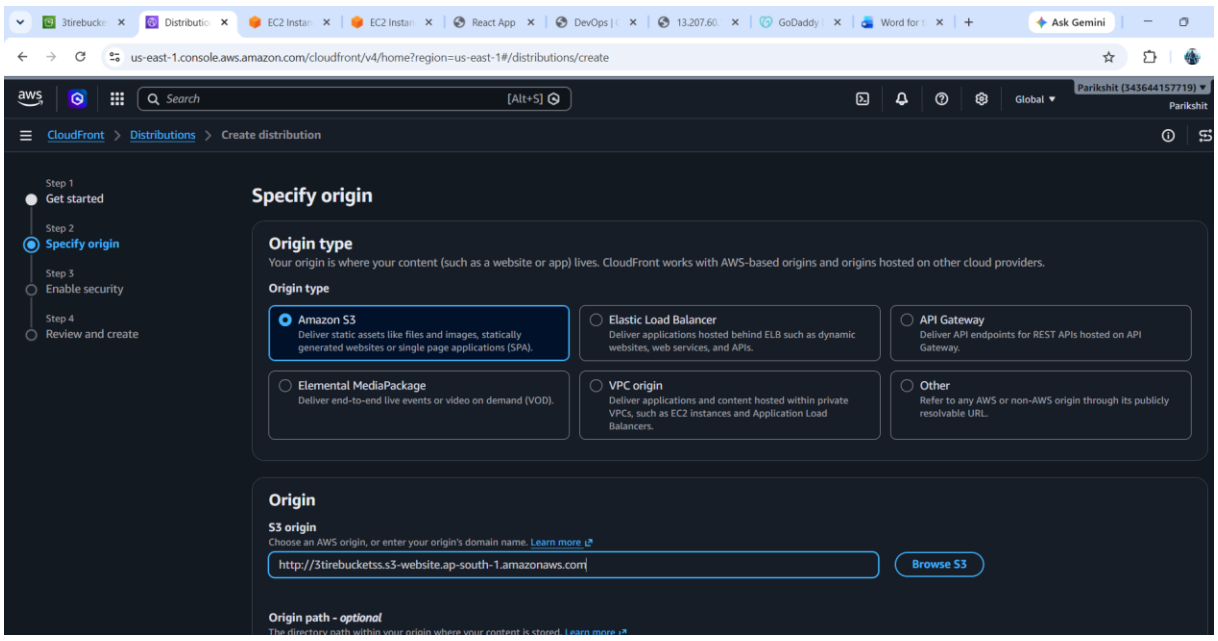


Figure 9.3 — Configuring a CloudFront distribution with the S3 website endpoint as origin.

10. End-to-End Testing & Verification

With every tier in place, the application was verified end-to-end: loading the frontend, submitting data through the API, and confirming it lands in RDS.

10.1 Frontend Landing Page

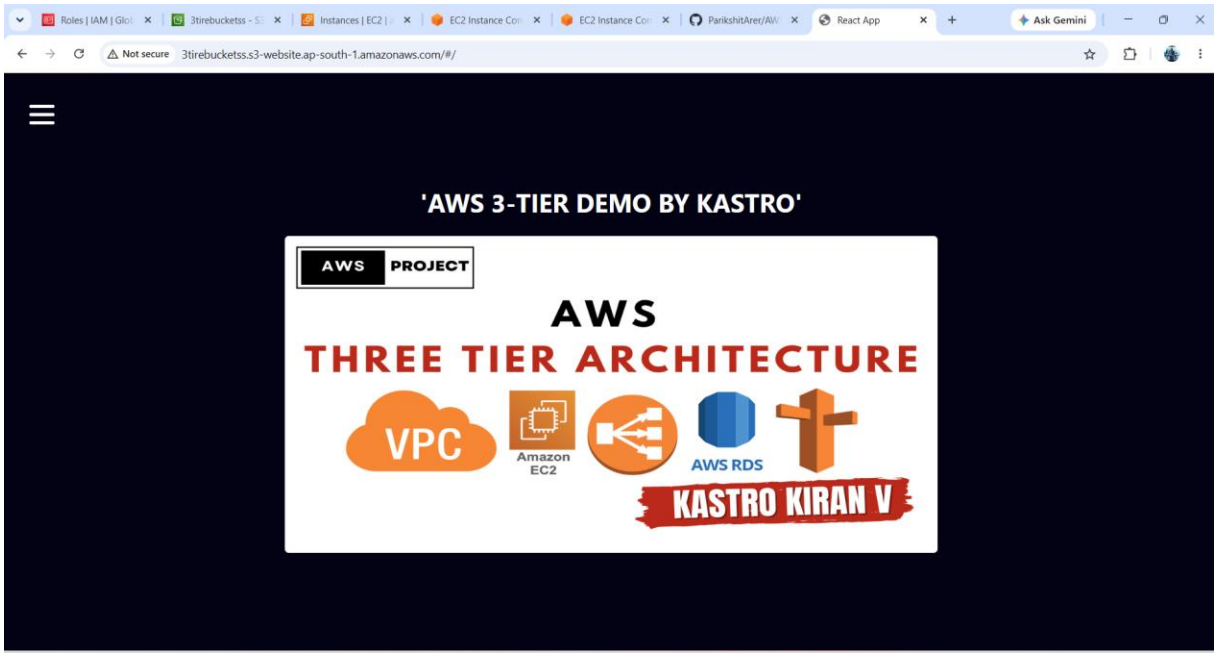


Figure 10.1 — The React frontend, served from the S3 static website endpoint.

10.2 Database Read/Write Demo

The "Aurora Database Demo" page exercises the full request path: the browser calls the EC2-hosted API, which reads from and writes to the RDS MySQL database. Rows added through the ADD button and removed through DEL persist correctly, confirming the application and database tiers are wired together correctly.

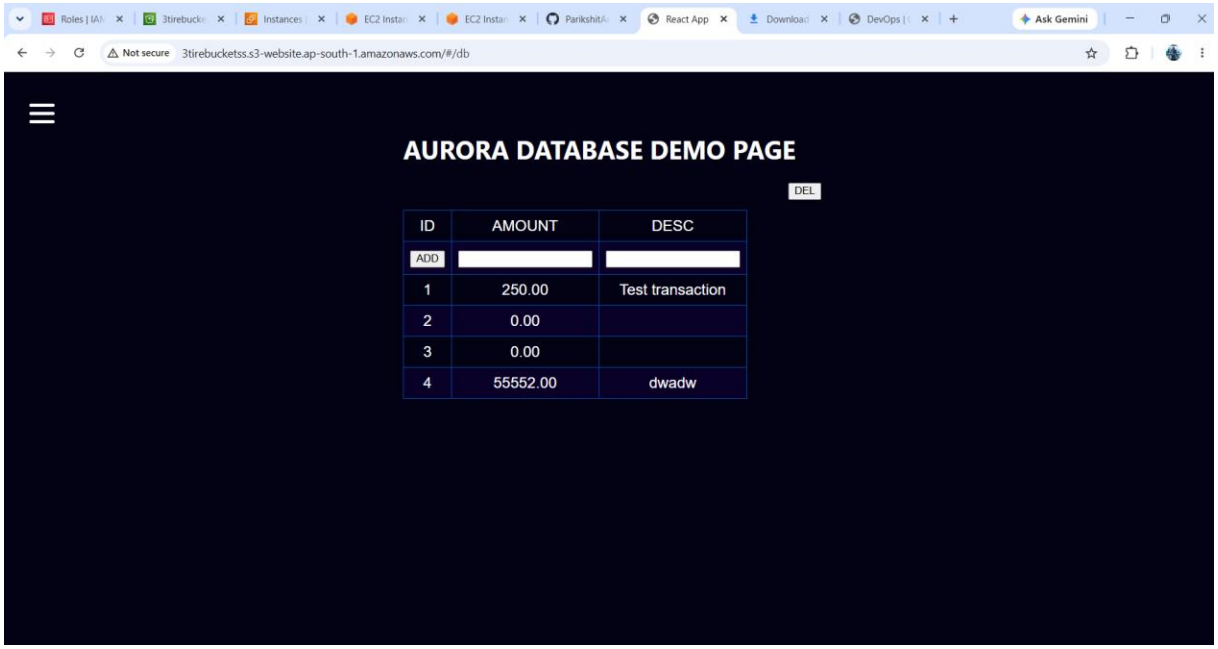


Figure 10.2 — Live transactions table, populated via the EC2 backend from data stored in RDS.

10.3 Custom Domain Access

Finally, the application is confirmed reachable directly at the custom domain configured in Route 53, with a secure connection in the browser.

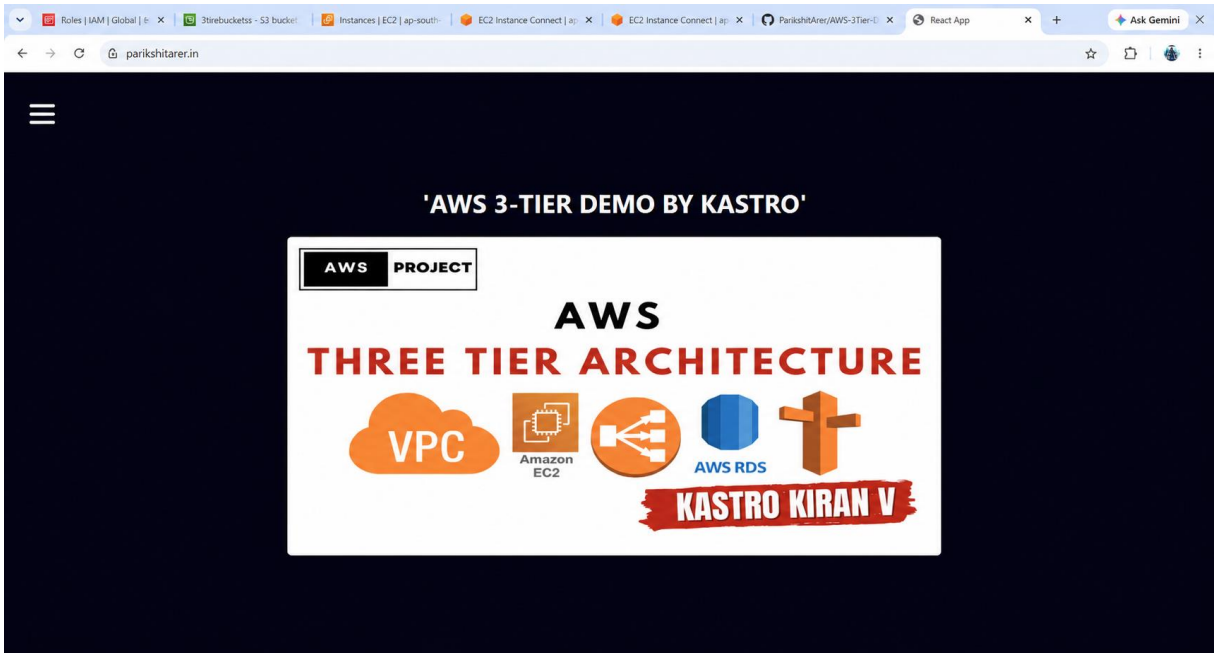


Figure 10.3 — The application live at parikshitarer.in, served over HTTPS via the custom domain.

11. Summary & Outcomes

This deployment shows that a fully functional, secure 3-tier web application can be assembled directly from native AWS services, with no orchestration tooling required:

✅Fully Functional 3-Tier Web App — users can submit data, which is stored in RDS.

✅Secure & Isolated Architecture — VPC, private subnets, and IAM roles ensure security.

✅Custom Domain Access — Route 53 provides a user-friendly URL, with ACM/CloudFront enabling HTTPS.

✅No DevOps Tools Used — entirely manual AWS setup, in contrast to the automated CI/CD approach used elsewhere.

Together, the S3-hosted frontend, the EC2 application server, and the RDS database — bound by VPC network isolation, scoped IAM roles, and Route 53 DNS — deliver on every business requirement set out at the start of the project: data submission, secure storage, custom-domain access, and adherence to AWS security best practices.